

gcache

- [1. 前言](#)
- [2. 特性](#)
- [3. 快速安装](#)
- [4. 简单举例](#)
- [5. 设置淘汰时间举例](#)
- [6. 其他算法举例](#)
 - [6.1. 最不经常使用 \(LFU\)](#)
 - [6.2. 最近使用最少的\(LRU\)](#)
 - [6.3. 自适应替换缓存\(ARC\)](#)
 - [6.4. 添加hanlder使用](#)

1. 前言

开发时，如果你需要对数据进行临时缓存，按照一定的淘汰策略，那么gcache你一定不要错过。gcache golang的缓存库。它支持可扩展的Cache，可以选择 LFU、LRU、ARC等淘汰算法。

2. 特性

gcache 有很多特性：支持过期淘汰算法Cache,比如 LFU, LRU和ARC

Goroutine安全

支持事件处理程序，淘汰、清除、添加 (可选)

自动加载缓存，如果它不存在 (可选)

.....

3. 快速安装

直接get即可使用。

```
go get -u https://github.com/bluele/gcache
```

4. 简单举例

```
package main

import (
    "github.com/bluele/gcache"
    "fmt"
)

func main() {
    gc := gcache.New(20).
        LRU().
        Build()
```

```
gc.Set("key", "ok")
value, err := gc.Get("key")
if err != nil {
    panic(err)
}
fmt.Println("Get:", value)
}
```

执行，控制台输出如下：

Get: ok

5. 设置淘汰时间举例

```
package main

import (
    "github.com/bluele/gcache"
    "fmt"
    "time"
)

func main() {
    gc := gcache.New(20).
        LRU().
        Build()
    gc.SetWithExpire("key", "ok", time.Second*10)
    value, _ := gc.Get("key")
    fmt.Println("Get:", value)

    // Wait for value to expire
    time.Sleep(time.Second*10)

    value, err := gc.Get("key")
    if err != nil {
        panic(err)
    }
    fmt.Println("Get:", value)
}
```

执行，控制台输出如下：

```
Get: ok
panic: Key not found.

goroutine 1 [running]:
main.main()
    /Users/laocheng/work/code/market-data-backend/utils/t/2.go:22
```

```
+0x21b
exit status 2
```

可以看到，一开始获取成功；但是超时时间设定后，过期删除，无法获取到了。

6. 其他算法举例

6.1. 最不经常使用 (LFU)

```
func main() {
    // size: 10
    gc := gcache.New(10).
        LFU().
        Build()
    gc.Set("key", "value")
}
```

6.2. 最近使用最少的(LRU)

```
func main() {
    // size: 10
    gc := gcache.New(10).
        LRU().
        Build()
    gc.Set("key", "value")
}
```

6.3. 自适应替换缓存(ARC)

- 在LRU和LFU之间不断平衡，以提高综合结果

```
func main() {
    // size: 10
    gc := gcache.New(10).
        ARC().
        Build()
    gc.Set("key", "value")
}
```

6.4. 添加handler使用

```
func main() {
    gc := gcache.New(2).
        AddedFunc(func(key, value interface{}) {
```

```
        fmt.Println("added key:", key)
    }).
    Build()
    for i := 0; i < 3; i++ {
        gc.Set(i, i*i)
    }
}
```

执行，控制台输出如下：

```
added key: 0
added key: 1
added key: 2
```

可以在set时候做一些额外的处理。